

Resumen 8

Gonzalo Balladares - 2262595J

2025-11-17

Web sockets

```
const Socket = require('koa-websocket');
const app = Socket(new Koa());
const wsRouter = new Router();
wsRouter.all('/ws', (ctx) => {
  ctx.websocket.on('message', (message) => {
    ctx.websocket.send(`Server received: ${message}`);
  });
  ctx.websocket.on('close', () => {
    console.log('WebSocket connection closed');
  });
});
```

Cliente

HTML DOM Document Object Model Es una interfaz que permite a los lenguajes de programación acceder y actualizar el contenido, la estructura y el estilo de un documento HTML. Está organizado como un árbol de objetos, donde cada nodo representa un elemento del documento (document, html element (body, head), element).

CSSOM

 CSS Object Model

Mapa de todos los selectores y propiedades relevantes para cada selector definido en CSS.

En conjunto con el DOM, forman el Render Tree (el “dibujo” de la página web).

```
<!DOCTYPE html>
<head>
  <title>Titulo</title>
</head>
<body>
  <h1>Hola Mundo</h1>
  <p>Este es un párrafo.</p>
</body>
</html>
```

```
/* selector de tipo */
p {
  color: blue;
}
/* selector de clase */
.class {
  font-size: 14px;
}
/* selector de ID */
#Id {
  margin: 10px;
}
/* selector de atributo */
input[type="text"] {
  border: 1px solid black;
}
/* selector descendiente */
div p {
  line-height: 1.5;
}
```

Modelo de encajonamiento: - Margin - Border - Padding - Content (Width, Height)

Event listeners Todo lo que está escuchando las interacciones con el usuario (clicks, movimientos del mouse, teclas, etc.)

```
elemento.addEventListener('click', function() {
  // código a ejecutar cuando se hace click
})
```

- click
- mouseover
- mouseout
- keydown
- keyup
- submit (cuando se envía un formulario)

REACT

Biblioteca de JS para construir UIs

El REACT DOM se encarga de actualizar el DOM sólo cuando el estado de un componente cambia, se actualiza sólo la parte del DOM que cambió, en vez de actualizar todo (Virtual DOM).

1. Initialization: se crea el componente.
2. Mounting: se inserta el componente en el DOM.
3. Updating: el componente se actualiza cuando cambian sus props o estado.
4. Unmounting: el componente se elimina del DOM.

```
// elementent.js
import ReactDOM from 'react-dom';

function Elementos() {
  const [elementos, setElementos] = useState([]);

  function armar(elemento) {
    return (
      <div className="mi-clase"> { /* className, no es HTML */}
      <h2>{elemento.titulo}</h2> { /* Todo lo que esta entre {} se evalua como JS */}
      <p>{elemento.descripcion}</p>
    </div>
  );
}
}
```

Hooks

Son funciones que permiten a los componentes de React para gestionar el estado y efectos secundarios - useState: permite añadir estado a un componente funcional, puede servir para manejar datos que cambian con el tiempo. - useEffect: permite gestionar efectos secundarios, renderizar algo después de que se cumpla una condición.

```
import React, { useState, useEffect } from 'react';
function Contador() {
  const [state, setState] = useState(0);
  useState(() => {
    // codigo de init
  });
  useEffect(() => {
    document.title = `Has hecho click ${state} veces`;
  } [state]);
}
```

VueJS

Framework de JS para construir UIs, se centra en la idea de Single Page Application (SPA, es una única página que cambia su contenido de forma dinámica con cambios desde el servidor, sin necesidad de refrescar la página por completo). Funciona con template (html con esteroides) y scripts (código de JS).

```
<template>
  <div>
    <input
      :value="text"
      @input="event => text = event.target.value"
    />
    <p>Message is: {{ message }}</p> { /* las vars de JS van entre {} */}
  </div>
</template>
<script>
  { /* codigo de JS */}
</script>
<style>
  { /* codigo de CSS */}
</style>
```

Estructura:

Name: Argument.Modifiers=Value

Names: - v-bind: vincula atributos de HTML a datos de Vue. (No se suele usar v-bind, se prefiere usar :) - v-model: crea un enlace bidireccional entre datos y elementos de formulario. - v-on: escucha eventos y ejecuta métodos. - v-if: renderiza condicionalmente elementos. - v-for: renderiza listas basadas en datos. - v-show: muestra u oculta elementos basados en una condición.

Arguments: - event: tipo de evento (click, input, submit, etc). (No se suelen usar eventos como v-on:event, se prefiere usar @event). - attribute: atributo HTML (href, src, class, style, etc). - key: clave única para listas renderizadas (usado con v-for). - model: propiedad vinculada en un componente (usado con v-model).

Modifiers: - .prevent: previene el comportamiento por defecto del evento. - .stop: detiene la propagación del evento. - .capture: usa la fase de captura del evento. - .self: el manejador solo se activa si el evento se origina en el elemento - .once: el manejador se activa solo una vez.

Value: - Puede ser una expresión JS, una variable, o el nombre de un método definido en la instancia de Vue.

* El browser va a hacer el default, más lo que uno le pide. El default puede generar sobreuso de recursos y brechas de seguridad. Prevent default:

```
event.preventDefault();
```

* El propagation es cuando un evento se “propaga” desde el elemento (div) hijo hasta los padres en el DOM. Detener la propagación evita que los manejadores de eventos en elementos padres se activen cuando un evento ocurre en un elemento hijo.

```
event.stopPropagation();
```